

Spending, Income and LAYS Analysis

This notebook examines how GDP per capita and public financing patterns are associated with Learning-Adjusted Years of Schooling (LAYS) across countries. It uses harmonized country-level inputs from prepared analysis files and compares multiple model specifications, including nonlinear and subgroup models. Final communication outputs are exported to Results/figures/ and Results/tables/.

Load Packages

```
#=====
# Import required packages
#=====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from matplotlib.patches import Rectangle, Ellipse
import seaborn as sns

import statsmodels.api as sm
import statsmodels.formula.api as smf

from pathlib import Path
import json
import sys
sys.path.append(r"C:\Users\F.Turner\Documents\00. Analyses")
import use_funcs
from use_funcs import find_project_root
```

Create Helper Functions

Define plotting palette and project path helpers used throughout the notebook.

```
#=====
# Define Save the Children colour palette
#=====
```

```
sc_colors = {'red': '#da291c',
             'medium red' : '#ed7b73',
             'light red': '#f9d3d0',
             'purple': '#ae90c3',
             'medium purple': '#cebcd9',
             'light purple': '#efe9f3',
             'yellow': '#fecf28',
             'medium yellow': '#ffeca9',
             'light yellow': '#fff5d4',
             'blue': '#99cccc',
             'medium blue' : '#d6ebeb',
             'light blue' : '#ebf5f5',
             'green' : '#45b283',
             'medium green' : '#8dd3b5',
             'light green' : '#d9f0e6',
             'grey' : '#e7e6e6'}
```

```
mpl.rcParams['font.family'] = 'Calibri'
```

```
#=====
# Configure project paths and output folders
#=====
```

```
PROJECT_ROOT = find_project_root(Path.cwd())
CONFIG_PATH = PROJECT_ROOT / "path_config.json"
```

```
with open(CONFIG_PATH, "r", encoding="utf-8") as f:
    PATHS = json.load(f)
```

```
IMP_DIR = (PROJECT_ROOT / PATHS["imports_dir"]).resolve()
HO_DIR = (PROJECT_ROOT / PATHS["handoff_dir"]).resolve()
EXP_DIR = (PROJECT_ROOT / PATHS["exports_dir"]).resolve()
FIG_DIR = (PROJECT_ROOT / PATHS["figures_dir"]).resolve()
TAB_DIR = (PROJECT_ROOT / PATHS["tables_dir"]).resolve()
```

```

for folder in [IMP_DIR, HO_DIR, EXP_DIR, FIG_DIR, TAB_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

print("PROJECT_ROOT:", PROJECT_ROOT)
print("IMP_DIR:", IMP_DIR)
print("HO_DIR:", HO_DIR)
print("EXP_DIR:", EXP_DIR)
print("FIG_DIR:", FIG_DIR)
print("TAB_DIR:", TAB_DIR)

```

```

PROJECT_ROOT: C:\Users\F.Turner\Documents\00. Analyses\Education Financing
IMP_DIR: C:\Users\F.Turner\Documents\00. Analyses\Education Financing\Data
HO_DIR: C:\Users\F.Turner\Documents\00. Analyses\Education Financing\Handoff
EXP_DIR: C:\Users\F.Turner\Documents\00. Analyses\Education Financing\Results
FIG_DIR: C:\Users\F.Turner\Documents\00. Analyses\Education Financing\Results\figures
TAB_DIR: C:\Users\F.Turner\Documents\00. Analyses\Education Financing\Results\tables

```

Load Data

Load the prepared merged datasets used in the modeling and visualization steps.

```

#=====
# Load core analysis datasets
#=====

all_data = pd.read_csv(IMP_DIR / "all_data.csv")
lays_panels = pd.read_csv(IMP_DIR / "lays_panels.csv")

```

Analysis

The following cells estimate the relationship between GDP per capita and LAYS, test alternative curve shapes, and assess differences by income group.

Does GDPPC Correlate with LAYS?

```
#=====
# Build country-level base dataset for GDPPC-LAYS models
#=====

df = all_data.copy()
df = df.dropna(subset=["lays", "gdppc_2015_usd"])
df = df.drop(df[df['income_group'] == 'Not classified'].index, axis=0)
df = df[["iso3",
        "year",
        "income_group",
        "lays",
        "gdppc_2015_usd"]]

df = df.groupby(["iso3"]).agg('first').reset_index()
```

What is the relationship between GDPPC and LAYS?

```
#=====
# Plot LAYS against GDPPC in level and log scale
#=====

fig, ax = plt.subplots(2, 1, figsize=(8, 12))

sns.scatterplot(data=df, x="gdppc_2015_usd",
                y="lays",
                hue="income_group",
                palette = {"Low income": sc_colors['green'],
                           "Lower middle income": sc_colors['red'],
                           "Upper middle income": sc_colors['purple'],
                           "High income": sc_colors['blue']},
                edgecolor='black',
                alpha=0.7,
                s=50,
                ax=ax[0])

ax[0].legend(title="Income Group", loc='upper left', bbox_to_anchor=(1, 1))

sns.scatterplot(data=df, x="gdppc_2015_usd",
                y="lays",
```

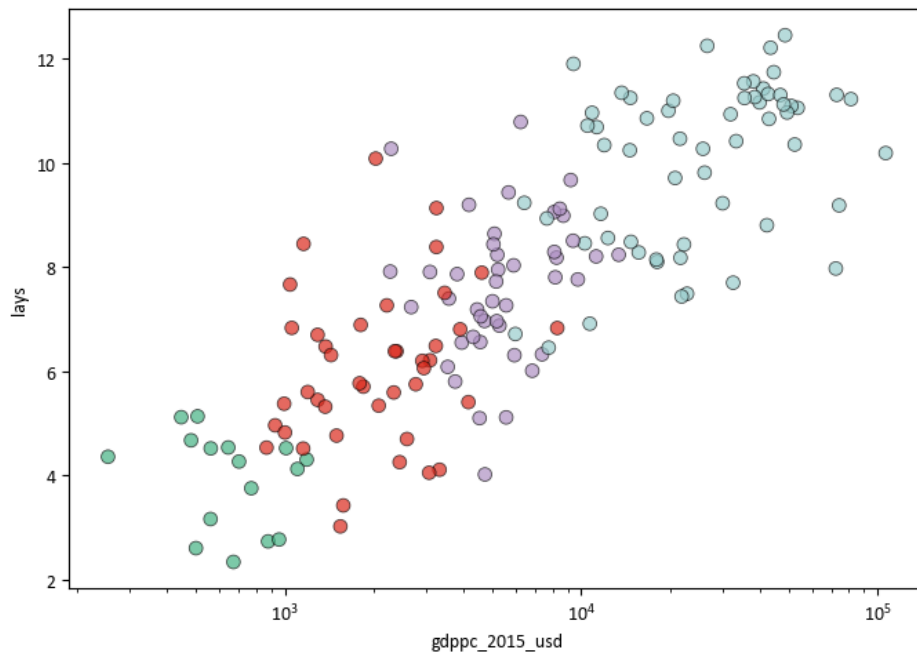
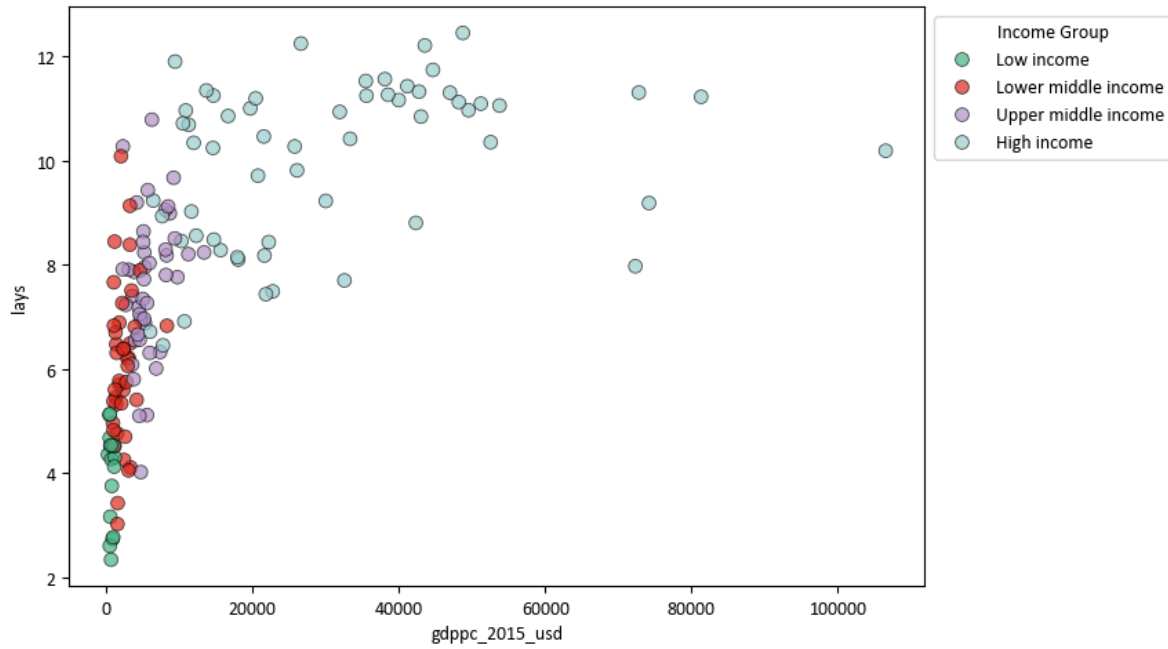
```

        hue="income_group",
        palette = {"Low income": sc_colors['green'],
                    "Lower middle income": sc_colors['red'],
                    "Upper middle income": sc_colors['purple'],
                    "High income": sc_colors['blue']},
        edgecolor='black',
        alpha=0.7,
        s=50,
        ax=ax[1])

ax[1].set_xscale("log")
ax[1].legend().set_visible(False)

fig.savefig(FIG_DIR / "lays_vs_gdppc_level_and_log.png", dpi=300, bbox_inches="tight")

```



```
#=====
# Estimate linear, log, and quartic GDPPC models
#=====
```

```

df1 = df.copy()

y = df1["lays"]

df1 = df1.rename(columns={"gdppc_2015_usd": "gdppc"})
df1['gdppc_s'] = (df1['gdppc'] - df1['gdppc'].mean()) / df1['gdppc'].std()

# Model 1. Linear Regression

X = df1[["gdppc_s"]]
X = sm.add_constant(X)
model_linear = sm.OLS(y, X).fit()

print("Model 1 - Linear Regression Summary:")
print(model_linear.summary())

# Model 2. Log-GDPPC Regression

df1['log_gdppc'] = np.log(df1['gdppc'])

X = df1[["log_gdppc"]]
X = sm.add_constant(X)
model_log = sm.OLS(y, X).fit()
print("\n\nModel 2 - Log-GDPPC Regression Summary:")
print(model_log.summary())

# 3. Quartic Regression

df1['gdppc2'] = df1['gdppc_s'] ** 2
df1['gdppc3'] = df1['gdppc_s'] ** 3
df1['gdppc4'] = df1['gdppc_s'] ** 4

X = df1[["gdppc_s", "gdppc2", "gdppc3", "gdppc4"]]

X = sm.add_constant(X)

model_quartic = sm.OLS(y, X).fit()

print("\n\nModel 3 - Quartic Regression Summary:")
print(model_quartic.summary())

```

Model 1 - Linear Regression Summary:

OLS Regression Results

```

=====
Dep. Variable:          lays    R-squared:                0.403
Model:                  OLS     Adj. R-squared:           0.399
Method:                 Least Squares    F-statistic:          107.4
Date:                  Mon, 30 Mar 2026    Prob (F-statistic):    1.50e-
19
Time:                  12:33:07    Log-Likelihood:        -
332.79
No. Observations:      161    AIC:                  669.6
Df Residuals:          159    BIC:                  675.7
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	7.7192	0.152	50.911	0.000	7.420	8.019
gdppc_s	1.5761	0.152	10.363	0.000	1.276	1.876

```

=====
Omnibus:                1.556    Durbin-Watson:          2.142
Prob(Omnibus):          0.459    Jarque-Bera (JB):        1.615
Skew:                  -0.228    Prob(JB):                0.446
Kurtosis:               2.817    Cond. No.:               1.00
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Model 2 - Log-GDPPC Regression Summary:

OLS Regression Results

```

=====
Dep. Variable:          lays    R-squared:                0.669
Model:                  OLS     Adj. R-squared:           0.667
Method:                 Least Squares    F-statistic:          322.1
Date:                  Mon, 30 Mar 2026    Prob (F-statistic):    4.60e-
40
Time:                  12:33:07    Log-Likelihood:        -
285.21
No. Observations:      161    AIC:                  574.4
Df Residuals:          159    BIC:                  580.6
Df Model:              1
Covariance Type:       nonrobust
=====

```


	coef	std err	t	P> t	[0.025	0.975]
const	-5.2522	0.732	-7.180	0.000	-6.697	-
3.807						
log_gdppc	1.4931	0.083	17.946	0.000	1.329	1.657
Omnibus:		0.568	Durbin-Watson:			2.127
Prob(Omnibus):		0.753	Jarque-Bera (JB):			0.383
Skew:		0.116	Prob(JB):			0.826
Kurtosis:		3.061	Cond. No.			57.7

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Model 3 - Quartic Regression Summary:

OLS Regression Results

Dep. Variable:	lays	R-squared:	0.654
Model:	OLS	Adj. R-squared:	0.645
Method:	Least Squares	F-statistic:	73.71
Date:	Mon, 30 Mar 2026	Prob (F-statistic):	5.83e-
35			
Time:	12:33:07	Log-Likelihood:	-
288.90			
No. Observations:	161	AIC:	587.8
Df Residuals:	156	BIC:	603.2
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	9.3971	0.225	41.729	0.000	8.952	9.842
gdppc_s	3.5383	0.249	14.210	0.000	3.046	4.030
gdppc2	-2.8596	0.432	-6.617	0.000	-3.713	-
2.006						
gdppc3	0.8451	0.229	3.684	0.000	0.392	1.298
gdppc4	-0.0820	0.031	-2.640	0.009	-0.143	-
0.021						
Omnibus:		0.948	Durbin-Watson:			2.101

Prob(Omnibus):	0.622	Jarque-Bera (JB):	0.641
Skew:	0.135	Prob(JB):	0.726
Kurtosis:	3.150	Cond. No.	246.

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
#=====
# Compare fitted linear, log, and quartic curves
#=====

# Create x grid
x_pred_s = np.linspace(df1["gdppc_s"].min(), df1["gdppc_s"].max(), 200)
x_pred_gdppc = x_pred_s * df1["gdppc"].std() + df1["gdppc"].mean()
x_pred_log = np.log(x_pred_gdppc)

X_pred_linear = sm.add_constant(pd.DataFrame({"gdppc_s": x_pred_s}),
                                   has_constant="add")
X_pred_log = sm.add_constant(pd.DataFrame({"log_gdppc": x_pred_log}),
                              has_constant="add")
X_pred_quartic = pd.DataFrame({
    "gdppc_s": x_pred_s,
    "gdppc2": x_pred_s ** 2,
    "gdppc3": x_pred_s ** 3,
    "gdppc4": x_pred_s ** 4,
})
X_pred_quartic = sm.add_constant(X_pred_quartic, has_constant="add")

y_linear = model_linear.predict(X_pred_linear)
y_log = model_log.predict(X_pred_log)
y_quartic = model_quartic.predict(X_pred_quartic)

fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(x_pred_s, y_linear,
        label=f"Linear (r2={model_linear.rsquared:.2f})",
        color=sc_colors['red'])
ax.plot(x_pred_s, y_log,
        label=f"Log-GDPPC (r2={model_log.rsquared:.2f})",
        color=sc_colors['purple'])
ax.plot(x_pred_s, y_quartic,
```

```

        label=f"Quartic (r2={model_quartic.rsquared:.2f})",
        color=sc_colors['blue'])

sns.scatterplot(data=df1, x="gdppc_s", y="lays", ax=ax,
                color=sc_colors['grey'], edgecolor='black', alpha=0.7, s=20)

ax.set_xlabel("Standardized GDPPC (gdppc_s)")
ax.set_ylabel("LAYS")
ax.legend()

fig.savefig(FIG_DIR / "fitted_curves_gdppc_lays.png", dpi=300, bbox_inches="tight")

```

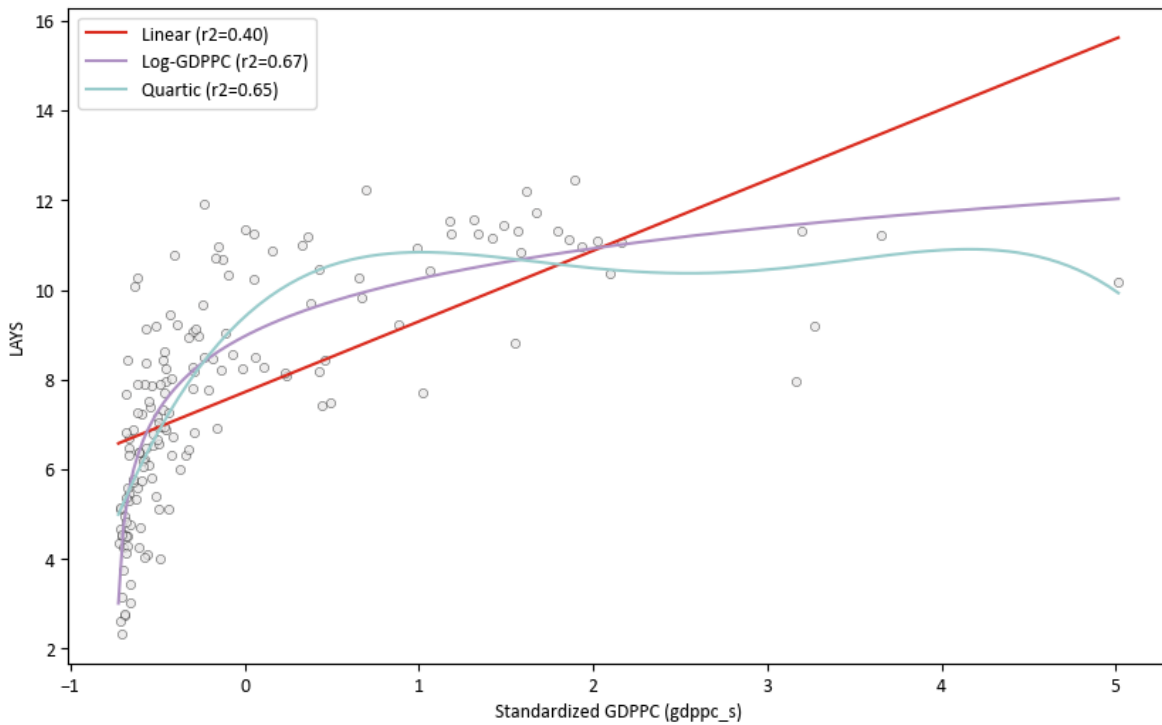


FIG 2: Fitting Curves for the relationship between GDP Per Capita (GDPPC) and LAYS

Here we fit three different curves to plot the relationship between GDPPC and LAYS. The first is a line, which is a relatively poor fit, explaining 43 percent of the variance in our standardised measure of GDPPC (standardised to allow for fitting of our quartic regression curve). Second, we plot the relationship between log-GDPPC and LAYS, which shows an initial steep relationship between GDPPC and LAYS, which becomes shallower for richer countries. Finally, mirroring Pritchett et al. (2026) we fit a quartic curve with linear, quadratic, cubic

and quartic coefficients. This explains slightly less variance than our log model, but shows a steep initial rise, before a slight dip, a slight rise, and a final dip for the richest countries.

Overall, our log model explains the most variance, and aligns with our theoretical expectation of diminishing returns on spending. Our next question is whether this relationship is consistent across income groups.

Do Slopes Vary by Income Group

```
#=====
# Estimate interaction model across income groups
#=====

# MODEL 4 - Log-GDPPC with Income Group Dummies and Interactions

df2 = df.copy()
df2 = df2.rename(columns={"gdppc_2015_usd": "gdppc"})
df2['log_gdppc'] = np.log(df2['gdppc'])

y = df2["lays"]

# Create income group dummies
dummies = pd.get_dummies(df2["income_group"], prefix="inc", drop_first=True)
df2 = pd.concat([df2, dummies], axis=1)

# Create interaction terms
for col in dummies.columns:
    df2[f"log_gdppc_{col}"] = df2["log_gdppc"] * df2[col]

# Now build X
X_vars = ["log_gdppc"] + [f"log_gdppc_{col}" for col in dummies.columns]

X = df2[X_vars]
X = X.astype(float) # Ensure all data is numeric
X = sm.add_constant(X)

model4 = sm.OLS(y, X).fit()
print("\n\nModel 4 - Log-GDPPC with Income Group Dummies and Interactions Summary:")
print(model4.summary())
```

Model 4 - Log-GDPPC with Income Group Dummies and Interactions Summary:

OLS Regression Results

```

=====
Dep. Variable:          lays    R-squared:                0.704
Model:                  OLS    Adj. R-squared:            0.696
Method:                 Least Squares    F-statistic:          92.58
Date:                   Mon, 30 Mar 2026    Prob (F-statistic):    3.58e-
40
Time:                   12:33:41    Log-Likelihood:        -
276.44
No. Observations:      161    AIC:                  562.9
Df Residuals:          156    BIC:                  578.3
Df Model:               4
Covariance Type:       nonrobust
=====

```

		coef	std err	t	P> t	[0.025
const		1.4419	1.752	0.823	0.412	-
2.020	4.903					
log_gdppc		0.8485	0.173	4.897	0.000	0.506
log_gdppc_inc_Low income		-0.4675	0.112	-4.186	0.000	-
0.688	-0.247					
log_gdppc_inc_Lower middle income		-0.2445	0.066	-3.721	0.000	-
0.374	-0.115					
log_gdppc_inc_Upper middle income		-0.1308	0.043	-3.048	0.003	-
0.216	-0.046					

```

=====
Omnibus:                0.225    Durbin-Watson:          2.259
Prob(Omnibus):          0.894    Jarque-Bera (JB):       0.088
Skew:                   -0.053    Prob(JB):               0.957
Kurtosis:               3.042    Cond. No.:              153.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

#=====
# Fit and plot separate log-GDPPC slopes by income group
#=====

```

```

## MODEL 5: Models per Income Group

df_lic = df2[df2['income_group'] == 'Low income']
df_lmic = df2[df2['income_group'] == 'Lower middle income']
df_umic = df2[df2['income_group'] == 'Upper middle income']
df_hic = df2[df2['income_group'] == 'High income']

fig, ax = plt.subplots(figsize=(10, 6))

sns.regplot(data=df_lic,
            x="log_gdppc",
            y="lays",
            label="LIC",
            scatter_kws={"color": sc_colors["medium green"],
                        "edgecolor": 'grey',
                        "alpha": 0.7},
            line_kws={"color": sc_colors["green"], "lw": 2},
            ax=ax, )
sns.regplot(data=df_lmic,
            x="log_gdppc",
            y="lays", ax=ax,
            label="LMIC",
            scatter_kws={"color": sc_colors["light red"],
                        'edgecolor': 'grey',
                        "alpha": 0.7},
            line_kws={"color": sc_colors["red"], "lw": 2})
sns.regplot(data=df_umic,
            x="log_gdppc",
            y="lays", ax=ax,
            label="UMIC",
            scatter_kws={"color": sc_colors["medium purple"],
                        'edgecolor': 'grey',
                        "alpha": 0.7},
            line_kws={"color": sc_colors["purple"], "lw": 2})
sns.regplot(data=df_hic,
            x="log_gdppc",
            y="lays", ax=ax,
            label="HIC",
            scatter_kws={"color": sc_colors["medium blue"],
                        'edgecolor': 'grey',
                        "alpha": 0.8},
            line_kws={"color": sc_colors["blue"], "lw": 2})

```

```

# Highlight visually notable clusters
ax.add_patch(Rectangle((6, 2), 1.0, 1.3, angle=0,
                        facecolor=sc_colors['grey'],
                        edgecolor=sc_colors['yellow'],
                        alpha=0.75, lw=0, zorder=0))

ax.add_patch(Rectangle((7, 7.4), 3.7, .75, angle=65,
                        facecolor=sc_colors['grey'],
                        edgecolor=None,
                        alpha=0.75, lw=1.8, zorder=0))

ax.legend(title="Income Group")
ax.set_xlabel("Log of GDP per Capita (2015 USD)")
ax.set_ylabel("LAYS")
ax.set_title("LAYS vs Log-GDPPC by Income Group",
             fontsize=16,
             fontname='Oswald')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

fig.savefig(FIG_DIR / "lays_vs_log_gdppc_by_income_group.png",
            dpi=300,
            bbox_inches="tight")

```

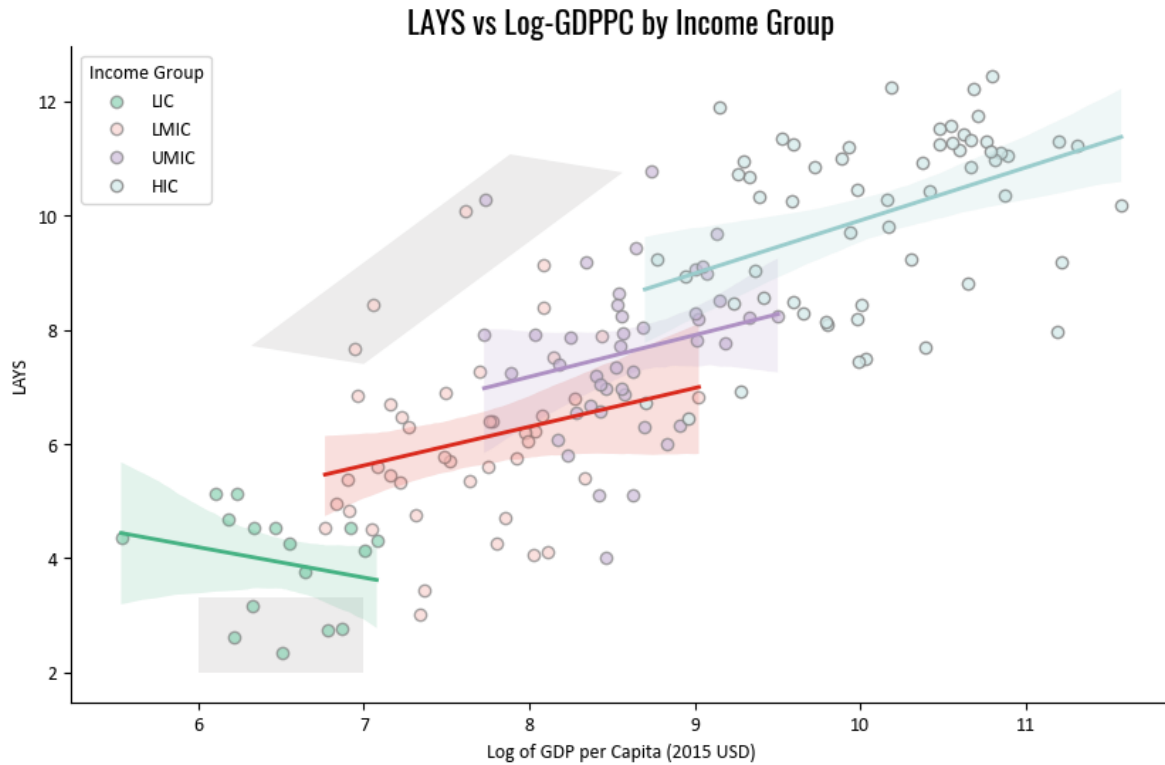


Fig 3: LAYS vs Log-GDPPC by Income Group

This figure shows the relationship between Log GDP Per Capita and LAYS per income group. While the overall relationship between log GDPPC and LAYS is strong and positive, when we dissect it we see that for Low-Income Countries the relationship is negative, and not statistically distinguishable from zero.

The implication of this is that, up to a certain threshold, increasing the amount of money in a system does not directly lead to improved institutions.

What is also notable, is that for LMICs and UMICs we have a number positive outliers, i.e., countries achieving stronger LAYS than would be expected for their income level. Conversely, for LICs there seem to be a cluster of countries achieving less than their income level would predict. We haven't validated this statistically, but it is visually apparent in the figure.

```
#####
# Estimate separate OLS models by income group
#####

# MODEL 5a - Low Income Countries

y = df_lic["lays"]
```



```

X = df_lic[["log_gdppc"]]
X = sm.add_constant(X)
model_5a = sm.OLS(y, X).fit()
print("\n\nModel 5a - Low Income Countries Summary:")
print(model_5a.summary())

# MODEL 5b - Lower Middle Income Countries
y = df_lmic["lays"]
X = df_lmic[["log_gdppc"]]
X = sm.add_constant(X)
model_5b = sm.OLS(y, X).fit()
print("\n\nModel 5b - Lower Middle Income Countries Summary:")
print(model_5b.summary())

# MODEL 5c - Upper Middle Income Countries
y = df_umic["lays"]
X = df_umic[["log_gdppc"]]
X = sm.add_constant(X)
model_5c = sm.OLS(y, X).fit()
print("\n\nModel 5c - Upper Middle Income Countries Summary:")
print(model_5c.summary())

# MODEL 5d - High Income Countries
y = df_hic["lays"]
X = df_hic[["log_gdppc"]]
X = sm.add_constant(X)
model_5d = sm.OLS(y, X).fit()
print("\n\nModel 5d - High Income Countries Summary:")
print(model_5d.summary())

```

Model 5a - Low Income Countries Summary:

OLS Regression Results

```

=====
Dep. Variable:          lays    R-squared:                0.054
Model:                  OLS     Adj. R-squared:             -
0.014
Method:                  Least Squares    F-statistic:          0.7937
Date:                    Mon, 30 Mar 2026    Prob (F-statistic):    0.388
Time:                    12:34:02    Log-Likelihood:        -
20.376

```

No. Observations: 16 AIC: 44.75
Df Residuals: 14 BIC: 46.30
Df Model: 1
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	7.3823	3.878	1.904	0.078	-0.935	15.700
log_gdppc	-0.5318	0.597	-0.891	0.388	-1.812	0.749
Omnibus:	2.877	Durbin-Watson:	3.054			
Prob(Omnibus):	0.237	Jarque-Bera (JB):	1.755			
Skew:	-0.584	Prob(JB):	0.416			
Kurtosis:	1.873	Cond. No.	112.			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Model 5b - Lower Middle Income Countries Summary:
OLS Regression Results

Dep. Variable:	lays	R-squared:	0.055			
Model:	OLS	Adj. R-squared:	0.032			
Method:	Least Squares	F-statistic:	2.338			
Date:	Mon, 30 Mar 2026	Prob (F-statistic):	0.134			
Time:	12:34:02	Log-Likelihood:	-			
74.632						
No. Observations:	42	AIC:	153.3			
Df Residuals:	40	BIC:	156.7			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	0.8722	3.383	0.258	0.798	-5.965	7.710
log_gdppc	0.6785	0.444	1.529	0.134	-0.218	1.575
=====						
Omnibus:	3.053	Durbin-Watson:	1.824			
Prob(Omnibus):	0.217	Jarque-Bera (JB):	1.968			
Skew:	0.476	Prob(JB):	0.374			
Kurtosis:	3.469	Cond. No.	116.			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Model 5c - Upper Middle Income Countries Summary:

OLS Regression Results

```

=====
Dep. Variable:          lays    R-squared:                0.046
Model:                  OLS    Adj. R-squared:            0.023
Method:                 Least Squares    F-statistic:        2.011
Date:                  Mon, 30 Mar 2026    Prob (F-statistic):    0.164
Time:                  12:34:03    Log-Likelihood:        -
74.794
No. Observations:      44    AIC:                  153.6
Df Residuals:          42    BIC:                  157.2
Df Model:              1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	1.3194	4.444	0.297	0.768	-7.648	10.287
log_gdppc	0.7319	0.516	1.418	0.164	-0.310	1.774

```

=====
Omnibus:                1.600    Durbin-Watson:        2.312
Prob(Omnibus):          0.449    Jarque-Bera (JB):      0.745
Skew:                  -0.035    Prob(JB):              0.689
Kurtosis:              3.634    Cond. No.               190.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Model 5d - High Income Countries Summary:

OLS Regression Results

```

=====
Dep. Variable:          lays    R-squared:                0.173
Model:                  OLS    Adj. R-squared:            0.159
Method:                 Least Squares    F-statistic:        11.96
Date:                  Mon, 30 Mar 2026    Prob (F-statistic):    0.00103
Time:                  12:34:03    Log-Likelihood:        -

```

103.39
 No. Observations: 59 AIC: 210.8
 Df Residuals: 57 BIC: 214.9
 Df Model: 1
 Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	0.6375	2.720	0.234	0.816	-4.810	6.085
log_gdppc	0.9276	0.268	3.458	0.001	0.391	1.465
Omnibus:	4.479	Durbin-Watson:	2.276			
Prob(Omnibus):	0.107	Jarque-Bera (JB):	3.165			
Skew:	-0.412	Prob(JB):	0.205			
Kurtosis:	2.221	Cond. No.	151.			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
#=====
# Compile subgroup coefficients into a summary table
#=====

coefficients = [(model_5a.params.to_list())[1], (model_5b.params.to_list())[1], (model_5c.params.to_list())[1]]
pvalues = [(model_5a.pvalues.to_list())[1], (model_5b.pvalues.to_list())[1], (model_5c.pvalues.to_list())[1]]
income_groups = ['Low Income', 'Lower Middle Income', 'Upper Middle Income', 'High Income']
models = ['model_5a', 'model_5b', 'model_5c', 'model_5d']
r2 = [eval(model).rsquared for model in models]

table = pd.DataFrame({
    'Model': models,
    'r2': r2,
    'Income Group': income_groups,
    'Coefficient': coefficients,
    'P-value': pvalues})

table = table.round(3)
table.to_csv(TAB_DIR / "income_group_log_gdppc_slopes.csv", index=False)

table
```

	Model	r2	Income Group	Coefficient	P-value
0	model_5a	0.054	Low Income	-0.532	0.388
1	model_5b	0.055	Lower Middle Income	0.679	0.134
2	model_5c	0.046	Upper Middle Income	0.732	0.164
3	model_5d	0.173	High Income	0.928	0.001

Other Drivers of Learning

```
#=====
# Prepare sample for additional-driver model
#=====

df = all_data.copy()

df = df.drop(df[df['income_group'] == 'Not classified'].index, axis=0)
df = df[["iso3",
        "year",
        "income_group",
        "lays",
        'fsi_total',
        'sap_share',
        'expenditure_pctbudget_uis',
        "gdppc_2015_usd"
        ]]

df = df.dropna()
```

```
#=====
# Restrict sample for additional-driver model
#=====

df = df.groupby("iso3").agg('first').reset_index()
df = df[(df['income_group'] == 'Low income') | (df['income_group'] == 'Lower middle income')]
df['log_gdppc'] = np.log(df['gdppc_2015_usd'])
```

```
<class 'pandas.DataFrame'>
Index: 47 entries, 0 to 123
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
#   :-----                                :-----
```

```

---      -----
0  iso3              47 non-null    str
1  year              47 non-null    int64
2  income_group      47 non-null    str
3  lays              47 non-null    float64
4  fsi_total          47 non-null    float64
5  sap_share          47 non-null    float64
6  expenditure_pctbudget_uis  47 non-null    float64
7  gdppc_2015_usd     47 non-null    float64
8  log_gdppc          47 non-null    float64

```

dtypes: float64(6), int64(1), str(2)

memory usage: 3.7 KB

```

#=====
# Fit multivariable OLS with additional controls
#=====

y = df['lays']

X = df[["log_gdppc", "fsi_total", "sap_share", 'expenditure_pctbudget_uis']]
X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

model.summary()

```

Dep. Variable:	lays	R-squared:	0.688
Model:	OLS	Adj. R-squared:	0.659
Method:	Least Squares	F-statistic:	23.19
Date:	Mon, 30 Mar 2026	Prob (F-statistic):	3.61e-10
Time:	13:12:29	Log-Likelihood:	-63.293
No. Observations:	47	AIC:	136.6
Df Residuals:	42	BIC:	145.8
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	8.7278	3.421	2.551	0.014	1.823	15.632
log_gdppc	0.3435	0.270	1.270	0.211	-0.202	0.889
fsi_total	-0.0145	0.019	-0.747	0.459	-0.054	0.025
sap_share	-37.2290	6.005	-6.200	0.000	-49.347	-25.111
expenditure_pctbudget_uis	4.4932	3.280	1.370	0.178	-2.126	11.112

Omnibus:	0.847	Durbin-Watson:	1.793
Prob(Omnibus):	0.655	Jarque-Bera (JB):	0.839
Skew:	0.297	Prob(JB):	0.657
Kurtosis:	2.724	Cond. No.	3.73e+03

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 3.73e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Education Expenditure

By COFOG Category and GDPPC

```
#=====
# Select COFOG expenditure variables
#=====

cofog = all_data[['iso3',
                  'year',
                  'gdppc_2015_usd',
                  'general_public_services_pct',
                  'defense_pct',
                  'public_order_safety_pct',
                  'economic_affairs_pct',
                  'environmental_protection_pct',
                  'housing_community_amenities_pct',
                  'health_pct',
                  'recreation_culture_religion_pct',
                  'education_pct',
                  'social_protection_pct']]

#=====
# Keep latest country-year COFOG observations
#=====

cofog = cofog.sort_values(['iso3', 'year'], ascending=[True, False])
cofog = cofog.groupby('iso3').first().reset_index()
cofog = cofog.dropna(subset=['education_pct'])
```

```

#=====
# Reshape COFOG data to long format
#=====

cofog_melt = cofog.melt(id_vars=['iso3', 'year', 'gdppc_2015_usd'],
                        value_vars=['general_public_services_pct',
                                    'defense_pct',
                                    'public_order_safety_pct',
                                    'economic_affairs_pct',
                                    'environmental_protection_pct',
                                    'housing_community_amenities_pct',
                                    'health_pct',
                                    'recreation_culture_religion_pct',
                                    'education_pct',
                                    'social_protection_pct'],
                        var_name='cofog_category',
                        value_name='percentage')

cofog_melt['cofog_category'] = cofog_melt['cofog_category'].str.replace('_pct', '').str.replace(' ', '_')

cofog_melt = cofog_melt.sort_values(['gdppc_2015_usd'], ascending=[True]).reset_index(drop=True)

#=====
# Plot COFOG expenditure composition by country
#=====

cofog_palette = {
    'Recreation Culture Religion': sc_colors['light yellow'],
    'Social Protection': sc_colors['light green'],
    'Economic Affairs': sc_colors['purple'],
    'Housing Community Amenities': sc_colors['medium blue'],
    'Defense': sc_colors['red'],
    'Public Order Safety': sc_colors['medium red'],
    'Health': sc_colors['medium green'],
    'Education': sc_colors['green'],
    'Environmental Protection': sc_colors['blue'],
    'General Public Services': sc_colors['grey'],
}

columns = ['iso3',
           'Education',
           'Health',

```



```

        'Social Protection',
        'Recreation Culture Religion',
        'Environmental Protection',
        'General Public Services',
        'Housing Community Amenities',
        'Economic Affairs',
        'Public Order Safety',
        'Defense']

fig, ax = plt.subplots(figsize=(24, 8))

df = cofog.sort_values(['gdppc_2015_usd'], ascending=[True]).copy()
df = df.drop(columns=['year', 'gdppc_2015_usd'])
df.columns = df.columns.str.replace('_pct', '').str.replace('_', ' ').str.title()
df.columns = columns
df = df.set_index('iso3')

plot_columns = [col for col in df.columns if col in cofog_palette]

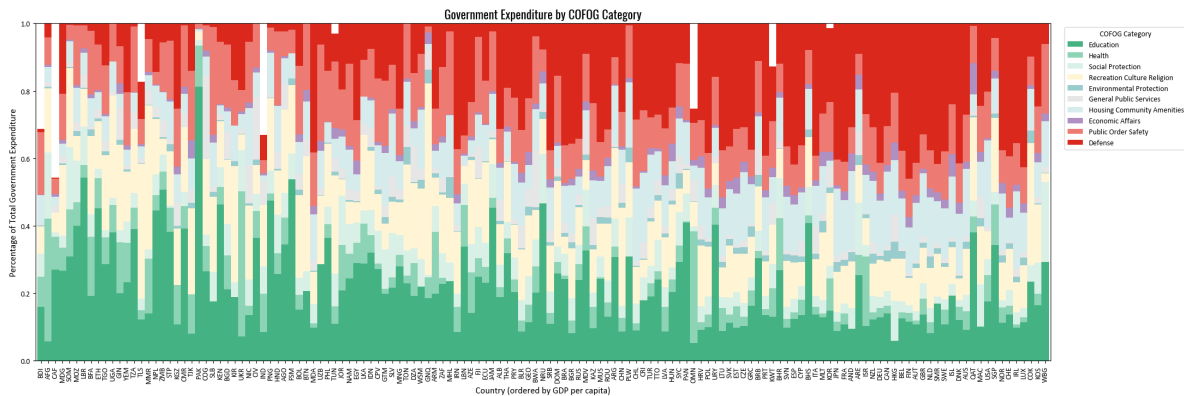
df.plot(
    kind='bar',
    stacked = True,
    color=[cofog_palette[col] for col in plot_columns],
    width=1,
    ax=ax,
)
ax.set_ylim(0, 1)

ax.set_xlabel('Country (ordered by GDP per capita)', fontsize=12)
ax.set_ylabel('Percentage of Total Government Expenditure', fontsize=12)

ax.set_title('Government Expenditure by COFOG Category', fontname='Oswald', fontsize=16)
ax.legend(title='COFOG Category', bbox_to_anchor=(1.01, 1), loc='upper left')

fig.savefig(FIG_DIR / "cofog_expenditure_composition.png", dpi=300, bbox_inches="tight")

```



Proportion of budget on education by GDPPC

```
#=====
# Plot change in education budget share against change in LAYS
#=====

df = lays_panels.copy()
df = lays_panels[lays_panels['panel'].isin([1, 2, 3, 4, 5])]

agg_map = {col: 'mean' for col in df.columns if col not in ['iso3', 'income_group']}
agg_map['income_group'] = 'first'
df = df.groupby('iso3', as_index=False).agg(agg_map)
# df = df[df['income_group'].isin(['Low income, Low middle income', 'Upper middle income', 'I

fig, ax = plt.subplots(figsize=(10, 6))

ax.add_patch(Rectangle((-0.03, -0.75), 0.03, 0.75, fill=True, color=sc_colors['light red'],

ax.add_patch(Rectangle((-0.03, 0), 0.03, 0.5, fill=True, color=sc_colors['light green'], alpha
ax.add_patch(Rectangle((0, 0), 0.025, 0.5, fill=True, color=sc_colors['medium green'], alpha
ax.add_patch(Rectangle((0, -0.75), 0.025, 0.75, fill=True, color=sc_colors['medium red'], al

ax.axhline(0, color='grey', linestyle='--')
ax.axvline(0, color='grey', linestyle='--')

sns.scatterplot(data=df, x='delta_expenditure_pctbudget_uis',
                y='delta_lays',
                hue='income_group',
```

```

# style='panel',
palette={'Low income': sc_colors['green'],
        'Lower middle income': sc_colors['medium red'],
        'Upper middle income': sc_colors['purple'],
        'High income': sc_colors['yellow'],
        'Not classified': sc_colors['grey']},

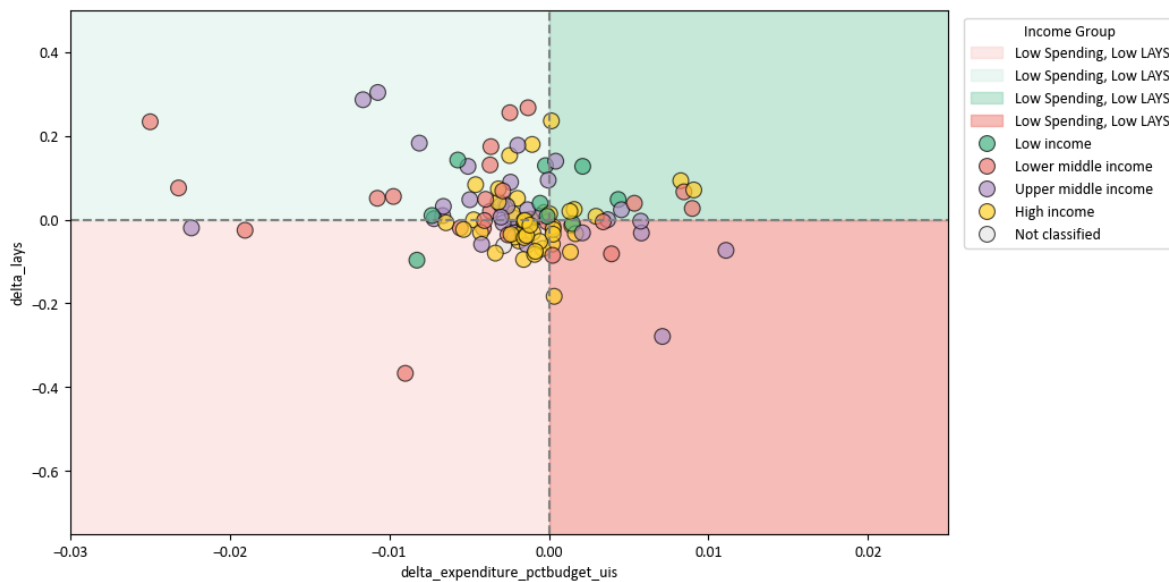
s=100,
edgecolor='black',
alpha=0.7,
ax=ax)

ax.set_xlim(-0.03, 0.025)
ax.set_ylim(-.75, .5)

ax.legend(title='Income Group', bbox_to_anchor=(1.01, 1), loc='upper left')

fig.savefig(FIG_DIR / "delta_education_budget_vs_delta_lays.png", dpi=300, bbox_inches="tight")

```



Exports

Important communication outputs are exported in the same cells where they are produced. Figures are saved to `Results/figures/` via `FIG_DIR`; summary tables are saved to

Results/tables/ via TAB_DIR.